

Tamper-Proof Audit Log Export — Design Spec

Date: 2026-07-06 **Status:** Draft **Sources:** NIST SP 800-53 (AU-9), OWASP auditability guidelines, RFC 5848, OpenBao Transit docs, GPT-5.5 oracle + reviewer subagents

1. Problem

The current audit log export (`GET /api/admin/audit-logs/export`) produces a plain CSV file with no integrity protection. A rogue SYSTEM_ADMIN could:

- Modify rows after export
- Reorder or delete rows
- Replace an export with a tampered version
- Deny the authenticity of a previously produced export

Third-party auditors have no way to independently verify that an export file has not been tampered with since creation.

2. Threat Model

Threat	Actor	Severity	Mitigation
Tamper with export file after creation	SYSTEM_ADMIN, external attacker	High	Hash-chain CSV + signed manifest
Replace newer export with older one	SYSTEM_ADMIN	Medium	Export registry + verifier warning
Intercept/modify in transit	External attacker	Medium	ZIP envelope + signature verification
Repudiate export authenticity	SYSTEM_ADMIN	High	Key-bound signature via OpenBao Transit
DB tampering before export	DB superuser	Out of scope	See §10

3. Design Overview

3.1 Format: Dual Export

Every secure export produces a ZIP containing three files. The ZIP itself is validated during extraction (see §3.3).

File	Format	Purpose	Verifiability
export.csv	Hash-chain CSV	Machine-readable audit data	Hash-chain recomputed from scratch
export.pdf	PDF (WeasyPrint)	Human-readable audit report	SHA256 compared against manifest
export.audit.sig	Signed JSON manifest	Cryptographic binding + metadata	OpenBao Transit verify

3.2 Data Flow

```

SYSTEM_ADMIN requests export with filter params
|
▼
GET /api/admin/audit-logs/export/secure?date_from=...&date_to=...
|
├ 1. INSERT INTO audit_export_registry (status='pending')
|   → returns export_id (monotonic SERIAL)
|
├ 2. Fetch rows from wims.system_audit_trails (same SQL, same filters)
|
├ 3. Generate hash-chain CSV (in-memory, canonical writer):
|   row_hash = SHA256(prev_hash || canonical_row_bytes)
|   where canonical_row_bytes = the CSV line WITHOUT the row_hash column,
|   with fixed column order, Unix \n newlines, UTF-8, consistent quoting
|
├ 4. Generate PDF via WeasyPrint (in-memory):
|   - BFP branding header
|   - Filter parameters displayed
|   - Export timestamp + export_id
|   - Table of audit rows (styled, paginated)
|   - Inline CSS only (no external resource loading)
|
├ 5. Compute SHA256 of:
|   - CSV bytes → csv_hash
|   - PDF bytes → pdf_hash
|   - CSV chain final row hash → csv_chain_final_hash
|
├ 6. Build canonical manifest JSON (fields in fixed order, no trailing comma):
|   {
|     "version": 1,
|     "export_id": 42,
|     "export_uuid": "a1b2c3d4-...",
|     "exported_at": "2026-07-06T06:30:00Z",
|     "exported_by": "SYSTEM_ADMIN (user_id: 7)",
|     "filters": {
|       "date_from": "2026-01-01",
|       "date_to": "2026-07-06",
|       "action_type": null,
|       "user_id": null
|     },
|   },

```

```

    "filter_hash": "sha256:abc...",
    "row_count": 1532,
    "csv_hash": "sha256:def...",
    "csv_chain_final_hash": "sha256:ghi...",
    "csv_dialect": {
        "delimiter": ",",
        "quoting": "MINIMAL",
        "encoding": "UTF-8",
        "newline": "LF",
        "columns": ["audit_id", "timestamp", "username", "action_type", ...]
    },
    "pdf_hash": "sha256:jkl...",
    "signing_key": {
        "provider": "openbao_transit",
        "key_name": "audit-export-signer",
        "key_version": 3,
        "algorithm": "sha2-256",
        "key_fingerprint": "sha256:mno..."
    }
}

└ 7. Strip signature field from manifest → serialize to canonical JSON
    → POST /v1/transit/sign/audit-export-signer { "input": base64(manifest_bytes) }
    → Receive signature

└ 8. Store signature in manifest.signature

└ 9. INSERT INTO audit_export_registry SET
    status='completed',
    csv_hash, pdf_hash, csv_chain_final_hash,
    signing_key_version=3,
    manifest_signature='bao:v1/...'

└ 10. Log AUDIT_SECURE_EXPORT action to system_audit_trails
    with record_id = export_id, result = 'completed'

└ 11. Return ZIP stream: export.csv + export.pdf + export.audit.sig

```

3.3 ZIP Security

The ZIP returned by the secure export endpoint must be validated during extraction:

- **Reject path traversal:** No filename containing `../` or absolute paths
- **Reject decompression bombs:** Reject if compression ratio exceeds 100:1 or uncompressed size exceeds 500MB
- **Reject duplicate filenames:** Exactly 3 files expected — `export.csv`, `export.pdf`, `export.audit.sig`. Any extra, missing, or duplicate files cause verification failure
- **Reject symlinks:** No symbolic links inside the ZIP

The verifier (both API and CLI) performs these checks before attempting any cryptographic verification.

3.4 Existing Endpoint Preserved

`GET /api/admin/audit-logs/export` remains unchanged for backward compatibility.

4. CSV Hash Chain Specification

4.1 Canonical CSV Writer

Every export uses a deterministic writer:

- **Column order:** fixed as defined in the manifest's `csv_dialect.columns`
- **Delimiter:** comma (,)
- **Quoting:** minimal (only when needed — commas, quotes, newlines in values)
- **Encoding:** UTF-8
- **Newlines:** Unix (`\n`) only
- **Header row:** included, hashed as row 0 (prev_hash for row 1)
- **Row hash column:** prepended as the first column, named `row_hash`

4.2 Hash Chain Formula

```
Row 0 (header):
  header_bytes = "audit_id,timestamp,username,...\n"
  prev_hash = SHA256(header_bytes)

Row N (data):
  row_data_bytes = the CSV line bytes WITHOUT the row_hash column
  row_hash = SHA256(hex(prev_hash) || row_data_bytes)
  full_row = hex(row_hash) + "," + row_data_bytes + "\n"
  prev_hash = row_hash

Final:
  csv_chain_final_hash = SHA256(hex(last_row_hash))
```

The `row_hash` column is **always the first column** in the CSV. When computing the hash for a row, the `row_hash` value and its leading comma are excluded from `row_data_bytes`. This avoids circular hashing.

4.3 Verification

The verifier: 1. Reads the CSV from the first row 2. Strips the `row_hash` column from each row 3. Recomputes the hash chain using the exact same formula 4. Compares the final hash against `manifest.csv_chain_final_hash`

5. PDF Specification

5.1 Generation

- **Library:** WeasyPrint (HTML → PDF)

- **Input:** Inline HTML with inline CSS only — no external stylesheet files
- **Content:** BFP header with logo, filter parameters, export metadata (export_id, timestamp), paginated table of audit rows, page numbers in footer
- **Security:** No external resource fetching — images must be base64-encoded inline or excluded. This prevents SSRF via HTML/CSS.

5.2 Verification

The verifier does **NOT** regenerate the PDF from data. Instead: 1. Compute SHA256 of the provided `export.pdf` bytes 2. Compare against `manifest.pdf_hash`

This avoids non-determinism from font versions, metadata, and rendering differences.

6. Manifest Format (Canonical JSON)

6.1 Signed Content

The signature covers the canonical JSON serialization of the manifest **without the signature field**. The JSON must be serialized with:

- Fixed key order (as listed in §3 step 6)
- No whitespace between keys/values
- No trailing commas
- UTF-8 encoding
- Use `\uXXXX` escaping for non-ASCII only when required by JSON spec

6.2 Signature Response

```
{
  "signature": "bao:v1/transit/sign/audit-export-signer:v3:abc123def456...",
  "signing_key_version": 3,
  "signed_at": "2026-07-06T06:30:00Z"
}
```

The `signature` field is a self-describing string: `bao:v1/{path}:{key_version}:{base64_signature}`.

7. Export Registry Table

```
CREATE TABLE IF NOT EXISTS wims.audit_export_registry (
  export_id          SERIAL PRIMARY KEY,
  export_uuid        UUID NOT NULL DEFAULT gen_random_uuid(),
  status             TEXT NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'completed', 'failed')),

  -- Who/what
  exported_by        UUID REFERENCES wims.users(user_id),
  exported_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```

-- Filter params (canonical JSON)
filter_json          JSONB NOT NULL,

-- Row metadata
row_count            INTEGER,
csv_hash             TEXT,    -- SHA256 of final CSV bytes
csv_chain_final_hash TEXT,    -- final hash of chain
pdf_hash             TEXT,    -- SHA256 of PDF bytes

-- Signing metadata
signing_key_name     TEXT,
signing_key_version  INTEGER,
manifest_signature   TEXT,    -- full signature string

-- Admin who triggered (for display, redundant with exported_by)
created_by_username  TEXT,

created_at           TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Index for "latest export for given filters" lookups
CREATE INDEX idx_audit_export_registry_created_at
ON wims.audit_export_registry (created_at DESC);

```

7.1 Replay/Staleness Checks

When the verifier runs **online** (via API endpoint): 1. Verify signature via OpenBao 2. Recompute CSV hash-chain, compare against registry 3. Check registry for newer completed exports with similar filters 4. If newer export exists → return **warning** alongside the integrity pass

When the verifier runs **offline** (CLI script with cached public key): 1. Verify signature using cached public key 2. Recompute CSV hash-chain 3. Cannot check for newer exports — result is "integrity verified (offline mode)" 4. Offline result includes a note: "Cannot verify this is the latest export — use the online verifier for freshness checks"

7.3 Offline Verifier Key Distribution

For air-gapped auditors who cannot reach OpenBao, the public key must be extracted once and distributed via secure channel:

```

# One-time export by a trusted OpenBao operator
bao read transit/keys/audit-export-signer
# → Extract the ECDSA P-256 public key from the response
# → Distribute the key fingerprint via secure channel (signed email, in-person)

```

The CLI verifier with `--offline` flag accepts the public key fingerprint as a hex string. Verification uses the Python `cryptography` library (already a dependency in the backend) for local ECDSA verification without contacting OpenBao.

8. OpenBao Signing Key

8.1 Key Setup

```
# Create dedicated signing key (one-time setup)
bao write -f transit/keys/audit-export-signer \
  type=ecdsa-p256 \
  deletion_allowed=false \
  exportable=false \
  allow_plaintext_backup=false
```

- **Type:** ECDSA P-256 (efficient, small signatures)
- **Not exportable:** Key material never leaves OpenBao
- **No plaintext backup:** Prevents key extraction
- **Key rotation:** Manual rotation via `bao write -f transit/keys/audit-export-signer/rotate`
- **Least privilege:** Backend service token limited to `transit/sign/audit-export-signer` and `transit/verify/audit-export-signer` only

8.2 Sign & Verify

```
# Sign (backend)
client = OpenBaoClient()
manifest_bytes = canonical_json_bytes(manifest_without_signature)
response = client._request("POST", "sign/audit-export-signer", {
    "input": base64.b64encode(manifest_bytes).decode(),
    "hash_algorithm": "sha2-256",
})
signature = response["data"]["signature"]
key_version = response["data"]["key_version"]

# Verify (verifier)
response = client._request("POST", "verify/audit-export-signer", {
    "input": base64.b64encode(manifest_bytes).decode(),
    "signature": signature,
    "hash_algorithm": "sha2-256",
})
is_valid = response["data"]["valid"] # True/False
```

8.3 Required Client Addition

Add `sign(key_name, data)` and `verify(key_name, data, signature)` methods to `src/backend/services/kms/openbao_client.py:OpenBaoClient`, following the same pattern as the existing `encrypt / decrypt` methods.

9. Verifier

9.1 API Endpoint

POST /api/admin/audit-logs/export/verify

Accepts multipart upload of the three files (CSV, PDF, .audit.sig). Returns:

```
{
  "verified": true,
  "warnings": ["A newer export (ID #47) exists for these filters"],
  "checks": {
    "signature": { "status": "pass", "key_name": "audit-export-signer", "key_version": 3 },
    "csv_hash_chain": { "status": "pass", "rows_verified": 1532 },
    "pdf_hash": { "status": "pass", "hash": "sha256:def..." },
    "registry": { "status": "pass", "export_id": 42, "matches": true },
    "freshness": { "status": "warn", "latest_export_id": 47, "message": "Newer export exists" }
  }
}
```

9.2 CLI Script

scripts/verify_audit_export.py

Usage:

```
# Online verification (requires OpenBao access)
python scripts/verify_audit_export.py export.csv export.pdf export.audit.sig

# Offline verification (uses cached public key fingerprint)
python scripts/verify_audit_export.py --offline --public-key-fingerprint <fingerprint> \
  export.csv export.pdf export.audit.sig
```

Dependencies: requests (online mode only), Python stdlib (hashlib, json, base64, csv).

Output:

```
Signature verified (key: audit-export-signer v3)
CSV hash-chain intact (1532 rows verified)
PDF hash matches manifest
Registry export #42 matches
Newer export #47 exists for these filters

INTEGRITY: PASS (with warnings)
```


10. Out of Scope

Item	Reason
DB tampering before export	This proves export package integrity, not database integrity at rest. DB-level tampering is mitigated by existing audit triggers (no-update/no-delete rules on <code>system_audit_trails</code>)
System clock manipulation	Mitigation requires an external timestamp authority (e.g., RFC 3161 TSA) — over-engineered for this phase
Real-time streaming export	Not needed — exports are point-in-time snapshots
Route geometry storage	Unrelated to audit export
ETA with time-of-day factor	Unrelated to audit export

11. Implementation Order

Phase 1 — Foundation

1. Add `sign()` and `verify()` methods to `OpenBaoClient` in `src/backend/services/kms/openbao_client.py`
2. Create `audit_export_registry` table migration (`src/postgres-init/`)
3. Add `AUDIT_SECURE_EXPORT` to audit action types

Phase 2 — Core Export

1. Implement canonical CSV writer with hash-chain in `src/backend/services/audit_export.py`
2. Implement PDF generation via WeasyPrint
3. Implement `/api/admin/audit-logs/export/secure` endpoint
4. Wire OpenBao signing into the export pipeline

Phase 3 — Verification

1. Implement `/api/admin/audit-logs/export/verify` API endpoint
2. Implement `scripts/verify_audit_export.py` CLI script
3. Add registry freshness check logic

Phase 4 — Tests

1. Unit tests for hash-chain CSV (correctness + edge cases)
2. Unit tests for manifest construction + signing round-trip
3. Integration test for full export → verify cycle
4. Integration test for tamper detection

12. Env Vars

Variable	Default	Purpose
OPENBAO_TRANSIT_MOUNT	transit	Existing — Transit engine mount path
WIMS_AUDIT_EXPORT_SIGNING_KEY	audit-export-signer	Signing key name

No new env vars needed — reuses existing OpenBao connection config (`OPENBAO_ADDR` , `OPENBAO_TOKEN`).